

COMPUTER ARITHMETIC

Module.4

CDA 3103C

Mesut Ozdag, Ph.D.

Background Information

- Binary Numbers
 - 2's Complement representation
 - Addition
 - Subtraction
- Arithmetic Logic Unit
 - Contains Adders to perform addition and subtraction

Integer Multiplication

- “Paper and pencil” example

Multiplicand	1000
Multiplier	$\times 1001$
	1000
	0000
	0000
	+ 1000
Product	01001000

Shift after each step

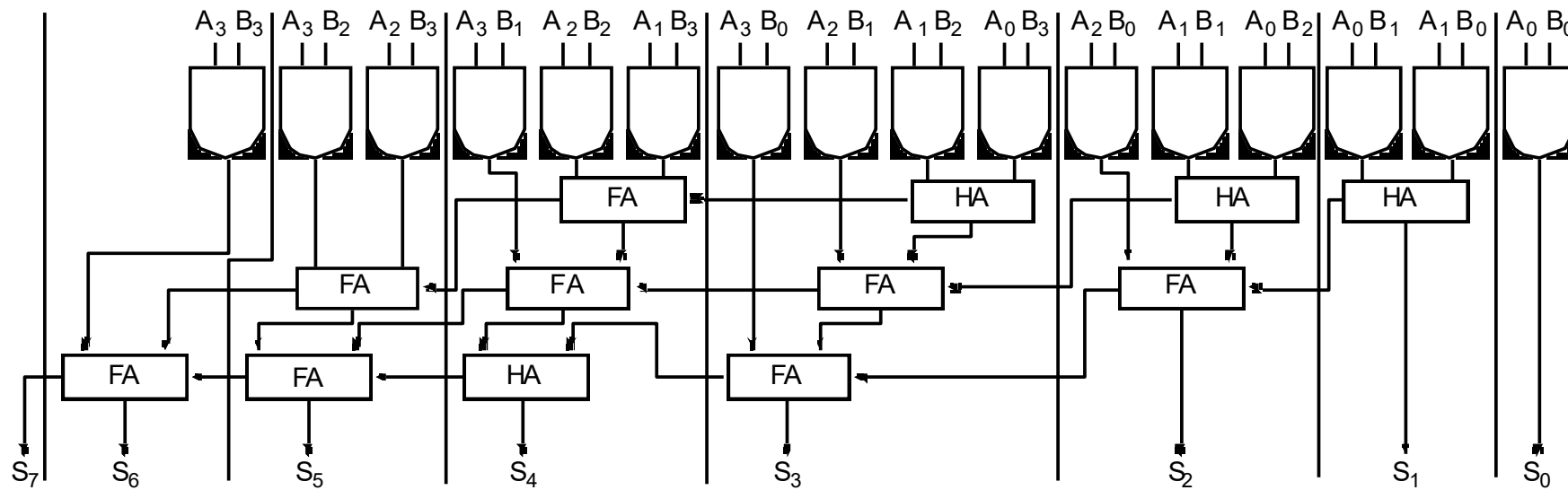
Combinational Multiplier

- Partial product accumulation

				A3	A2	A1	A0
				B3	B2	B1	B0
				A3 B0	A2 B0	A1 B0	A0 B0
			A3 B1	A2 B1	A1 B1	A0 B1	
		A3 B2	A2 B2	A1 B2	A0 B2		
	A3 B3	A2 B3	A1 B3	A0 B3			
S7	S6	S5	S4	S3	S2	S1	S0

Combinational Multiplier

- Partial product accumulation



Note use of parallel carry-outs to form higher order sums

Multiplication

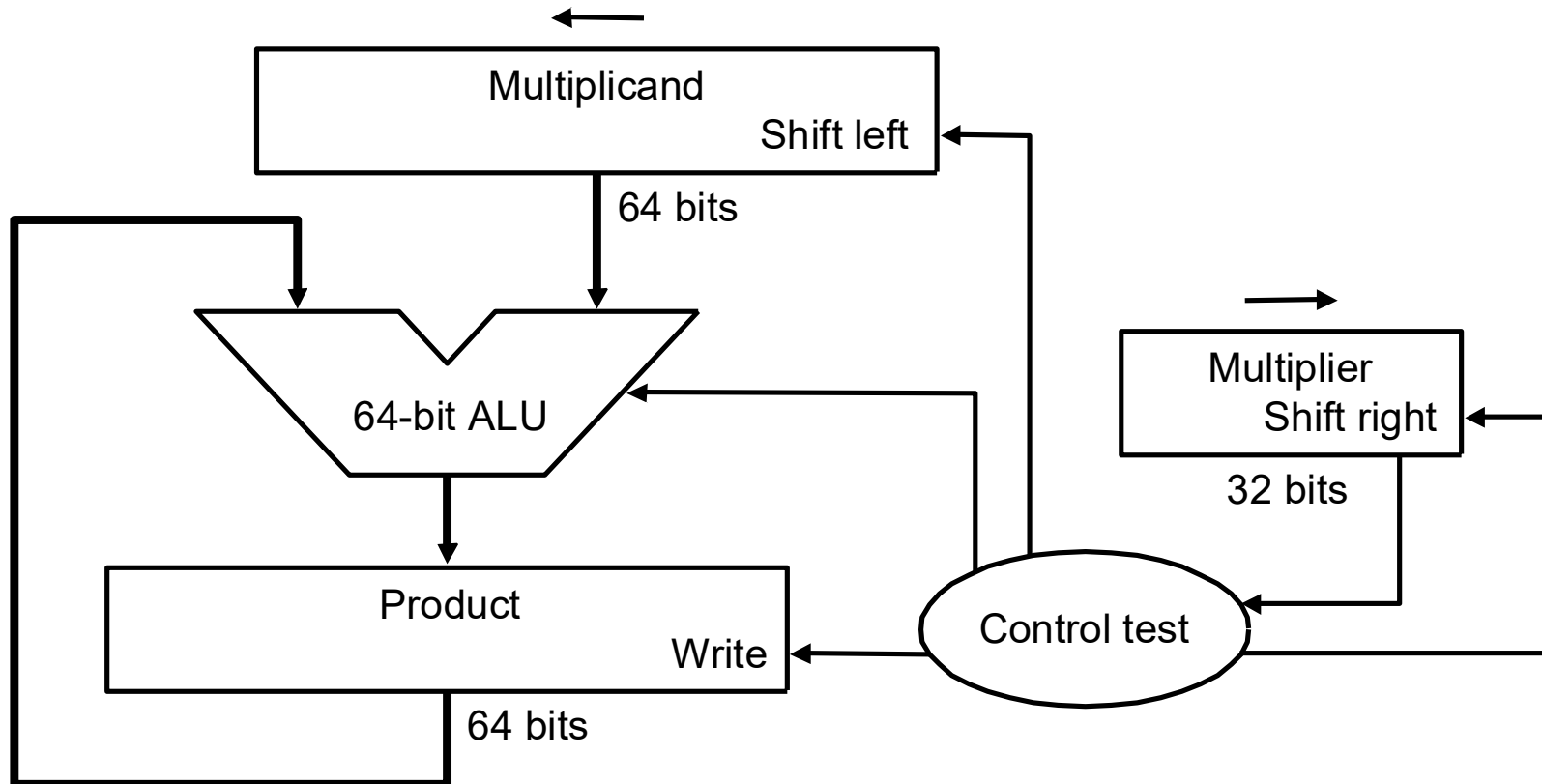
- Insight from paper and pencil algorithm
 - Shift the multiplicand left one digit each step
 - With 32 steps in a 32-bit number, we move 32 bits to the left
 - Requires a 64-bit register
 - Place 32 zeroes in the left half (unoccupied half)
 - Unsigned numbers do not require sign extension
- Multiplicand will be added to the sum in the product register
 - Product register will also be 64 bits
 - Requires a 64-bit ALU to add

Observations

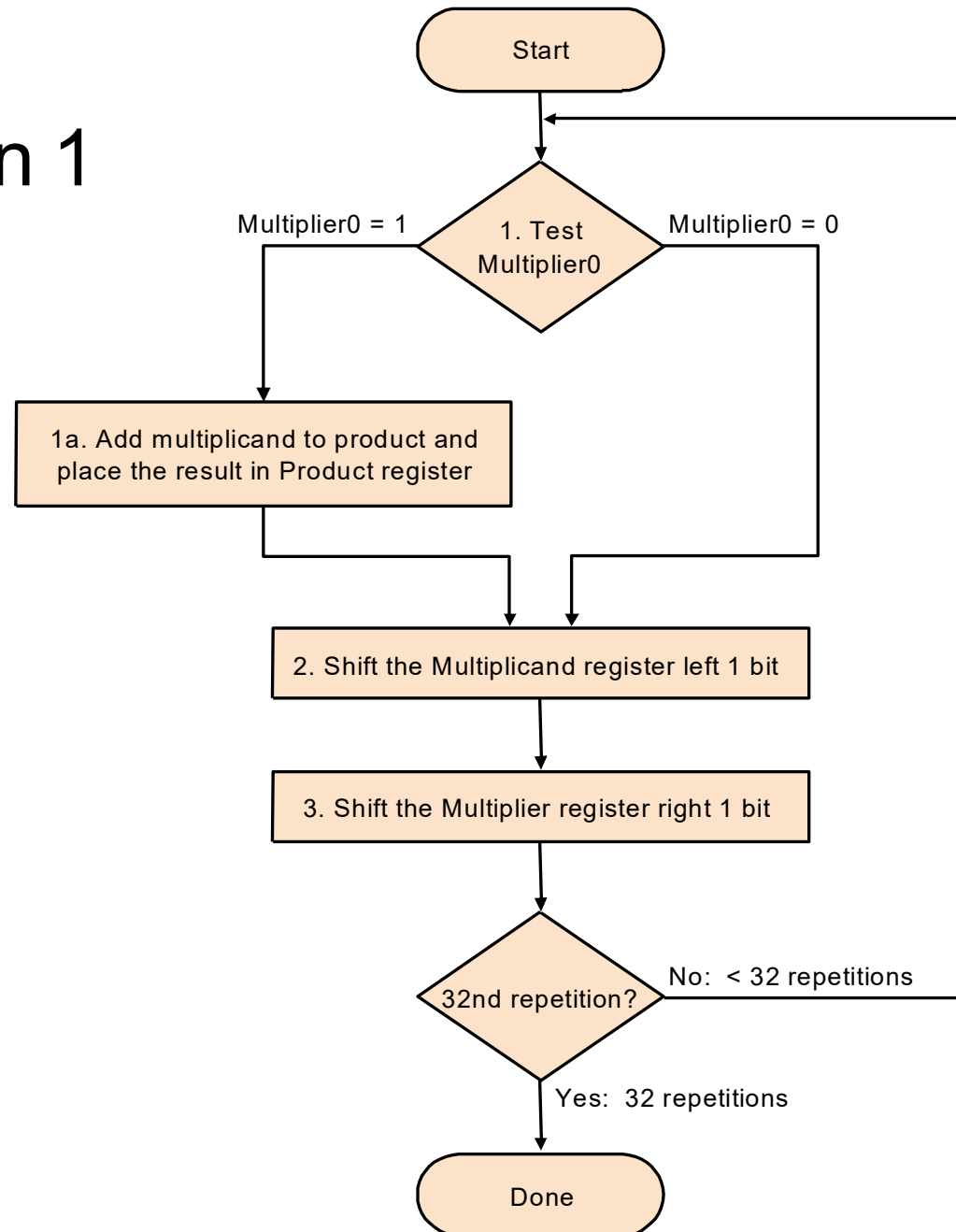
- Number of bits in the product is larger than the number in either the multiplicand or the multiplier.
 - m bits $\times$ n bits = $m+n$ bit product
 - Overflow is a possible issue
- Binary rules – “choices”
 - 0 $\Rightarrow$ place 0 (0 x multiplicand)
 - 1 $\Rightarrow$ place a copy (1 x multiplicand)
- 3 versions of unsigned multiplication hardware
 - successive refinement

Multiplication Hardware Version 1

- 64-bit Multiplicand reg, 64-bit ALU, 64-bit Product reg, 32-bit multiplier reg



Multiplication Algorithm Version 1



Multiplication Example (11x9)

Iter.	Step	Product	Multiplicand	Multiplier	Action
0	0	0000 0000	0000 1011	100 <u>1</u>	Initialize
1	1a.	0000 1011	0000 1011	100 <u>1</u>	Add
1	2,3	0000 1011	0001 0110	010 <u>0</u>	Shifts
2	1	0000 1011	0001 0110	010 <u>0</u>	Test-no add
2	2,3	0000 1011	0010 1100	001 <u>0</u>	Shifts
3	1	0000 1011	0010 1100	001 <u>0</u>	Test-no add
3	2,3	0000 1011	0101 1000	000 <u>1</u>	Shifts
4	1a.	0110 0011	0101 1000	000 <u>1</u>	Add
4	2,3	0110 0011	1011 0000	0000	Shifts

Multiplication is Time Consuming

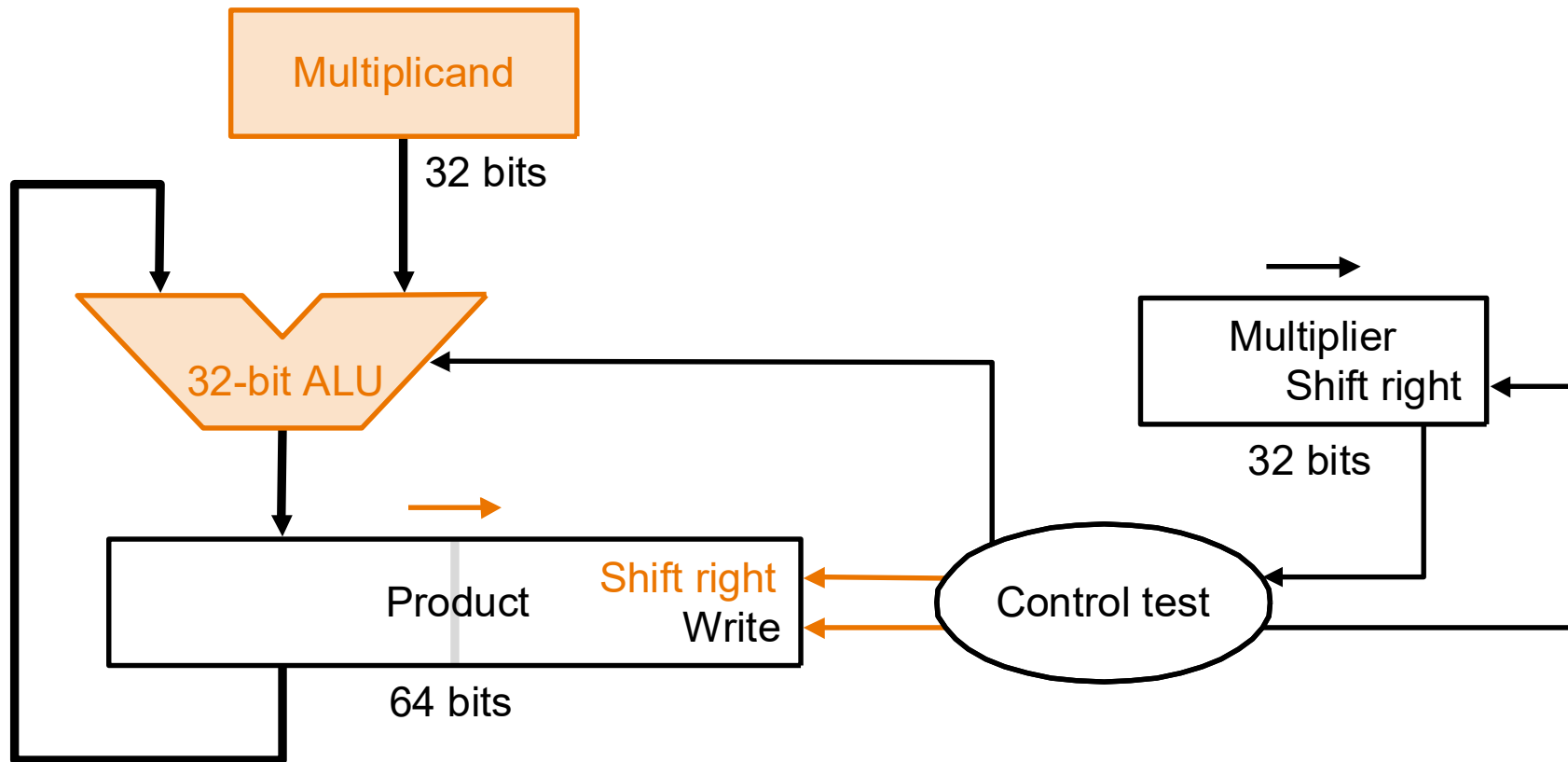
- 3 steps per iteration
- 32 iterations
- 96 steps total
- At 1 clock per step => ~ 100 clocks per multiply

Observations on Multiplication Version 1

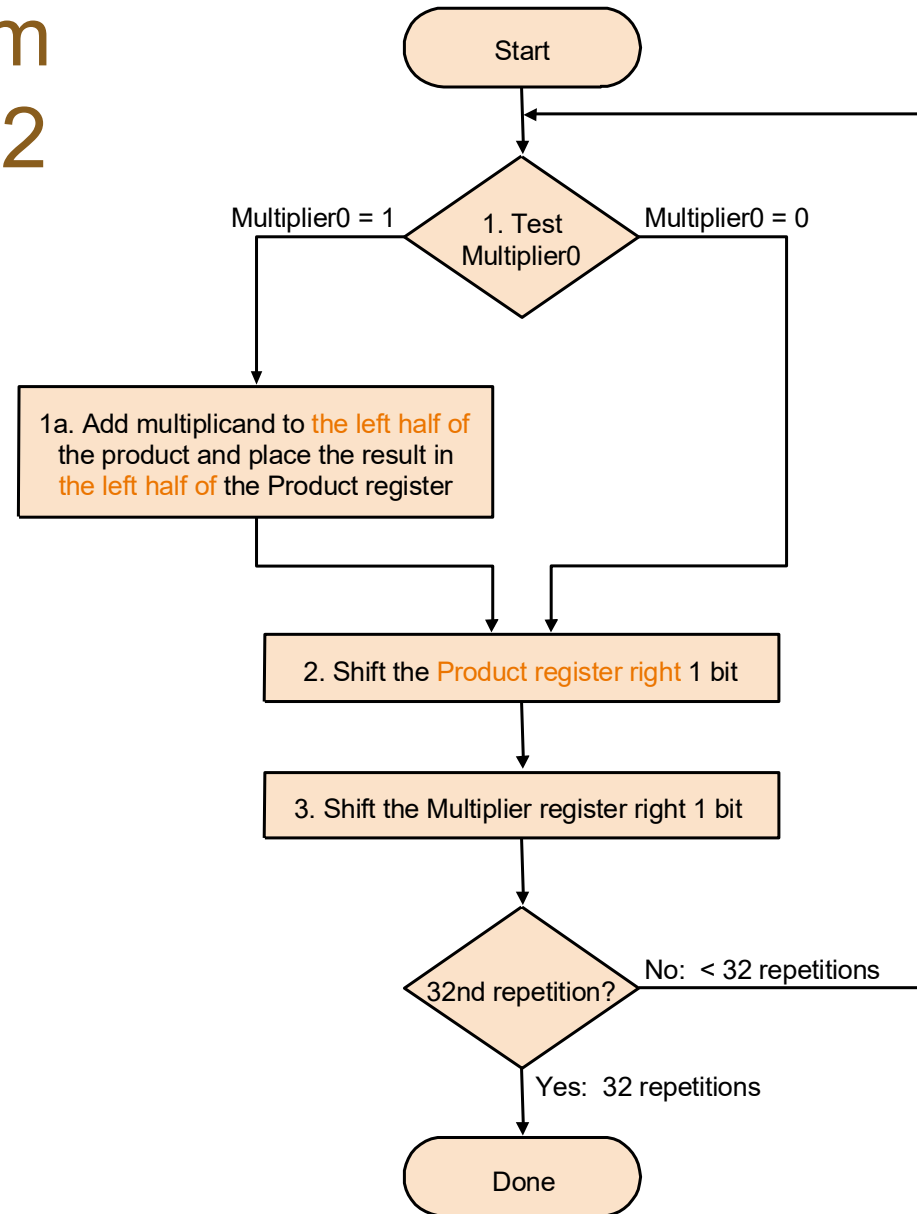
- Half the bits of the multiplicand are always 0
 - 64-bit adder is wasted
- 0's inserted in right of multiplicand as shifted
 - LSBs of product never changed once formed
- Instead of shifting the multiplicand to the left we can shift the product to the right
 - Perform some steps in parallel

Multiplication Hardware Version 2

- 32-bit Multiplicand reg, 32-bit ALU, 64-bit Product reg, 32-bit Multiplier reg



Multiplication Algorithm Version 2



Multiplication Example (11x9)

<u>Iter.</u>	<u>Step</u>	<u>Product</u>	<u>Multiplicand</u>	<u>Multiplier</u>	<u>Action</u>
0	0	00000000	1011	1001	Initialize

Multiplication Example (11x9)

<u>Iter.</u>	<u>Step</u>	<u>Product</u>	<u>Multiplicand</u>	<u>Multiplier</u>	<u>Action</u>
0	0	00000000	1011	1001	Initialize
					Test the LSB of multiplier
					1 indicates Add

Multiplication Example (11x9)

Iter.	Step	Product	Multiplicand	Multiplier	Action
0	0	00000000	1011	1001	Initialize
				1001	Add
					Add the left half of the product to the multiplicand.
					Store in left half of product.

Multiplication Example (11x9)

Iter.	Step	Product	Multiplicand	Multiplier	Action
0	0	00000000	1011	1001	Initialize
1	1a.	10110000	1011	1001	Add

Add the left half of the product to the multiplicand.
Store in left half of product.

Multiplication Example (11x9)

Iter.	Step	Product	Multiplicand	Multiplier	Action
0	0	00000000	1011	1001	Initialize
1	1a.	10110000	1011	1001	Add
Shift both the product and the multiplier to the right.					

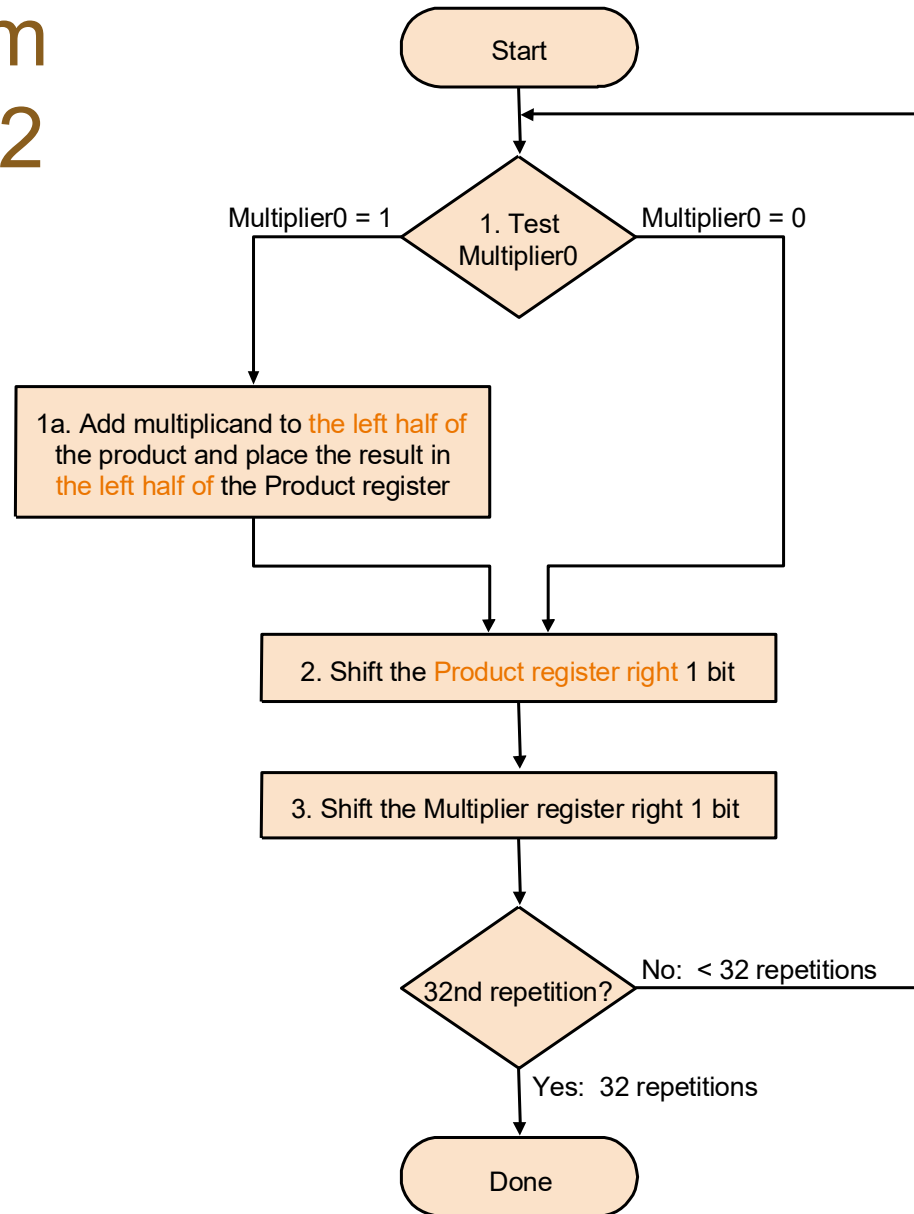
Multiplication Example (11x9)

<u>Iter.</u>	<u>Step</u>	<u>Product</u>	<u>Multiplicand</u>	<u>Multiplier</u>	<u>Action</u>
0	0	00000000	1011	1001	Initialize
1	1a.	10110000	1011	1001	Add
1	2, 3	01011000	1011	0100	Shifts Shift both the product and the multiplier to the right.

Multiplication Example (11x9)

Iter.	Step	Product	Multiplicand	Multiplier	Action
0	0	00000000	1011	1001	Initialize
1	1a.	10110000	1011	1001	Add
1	2,3	01011000	1011	0100	Shifts
2	1	01011000	1011	0100	Test-no add
2	2,3	00101100	1011	0010	Shifts
3	1	00101100	1011	0010	Test-no add
3	2,3	00010110	1011	0001	Shifts
4	1a.	11000110	1011	0001	Add
4	2,3	01100011	1011	0000	Shifts

Multiplication Algorithm Version 2



Observation

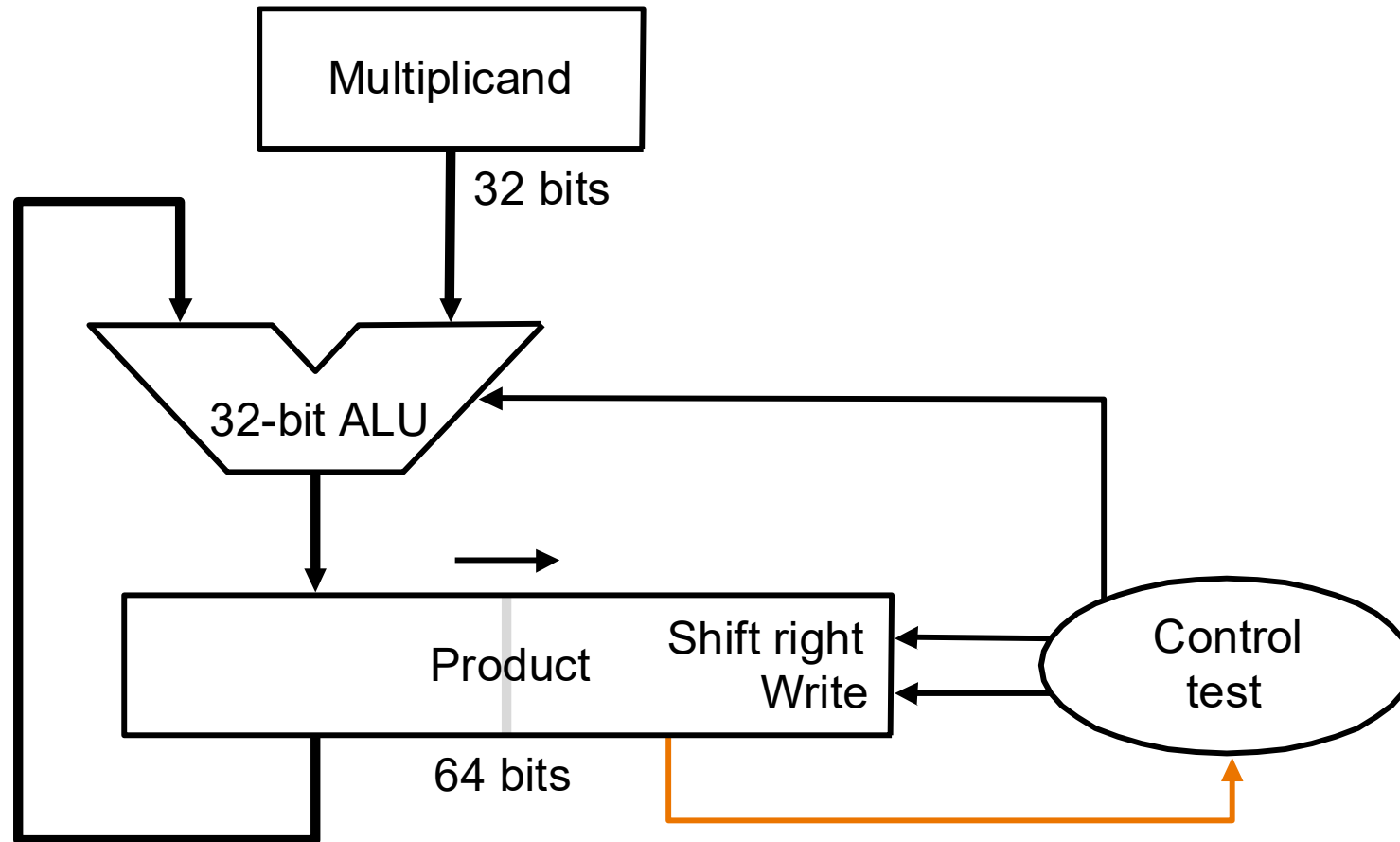
Product register
wastes space
(lower half = 0)

Exactly equal to the
size of multiplier left

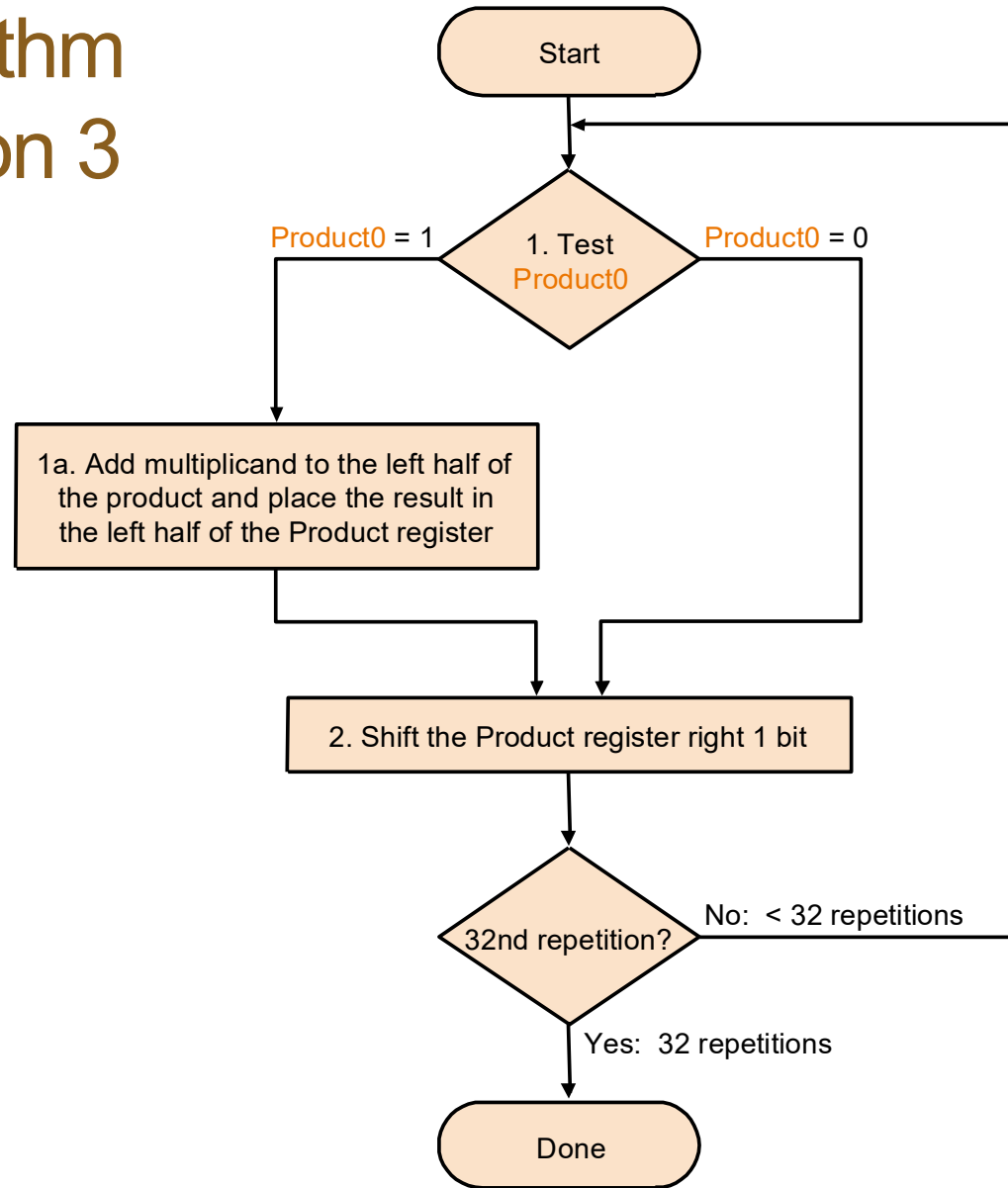
We can combine
Multiplier register
and Product register

Multiplication Hardware Version 3

- 32-bit Multiplicand reg, 32-bit ALU, 64-bit Product reg, (no Multiplier reg)



Multiplication Algorithm Version 3



Multiplication Example (11x9)

<u>Iter.</u>	<u>Step</u>	<u>Product</u>	<u>Multiplicand</u>	<u>Action</u>
0	0	0000 <u>1001</u>	1011	Initialize
1	1a.	1011 <u>1001</u>	1011	Add
1	2	0101 <u>1100</u>	1011	Shift
2	1	0101 <u>1100</u>	1011	Test-no add
2	2	0010 <u>1110</u>	1011	Shift
3	1	0010 <u>1110</u>	1011	Test-no add
3	2	0001 <u>0111</u>	1011	Shift
4	1a.	1100 <u>0111</u>	1011	Add
4	2	0110 0011	1011	Shift

Note: Multiplier in Product Register is underlined

Multiplying by a Constant

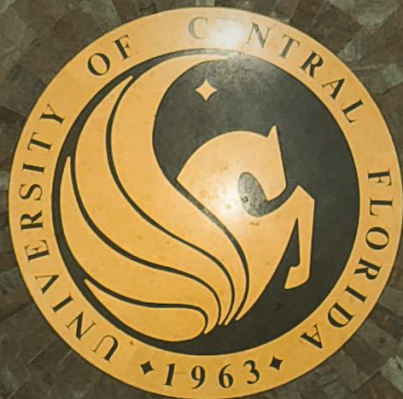
- Some compilers replace multiplies by short constants with a series of shifts and adds. Because one bit to the left represents a number twice as large in base 2, shifting the bits left has the same effect as multiplying by a power of 2.
- Almost every compiler will perform the strength reduction optimization of substituting a left shift for a multiply by a power of 2.

Multiplying by a Constant

- Some compilers replace multiplies by short constants with a series of shifts and adds. Because one bit to the left represents a number twice as large in base 2, shifting the bits left has the same effect as multiplying by a power of 2.
 - Almost every compiler will perform the strength reduction optimization of substituting a left shift for a multiply by a power of 2.
-
- $4 * 2 = 8$
 - $0100 * 0010 = 1000$
 - $0100 \ll 1 = 1000$

Multiplying by a Constant

- Some compilers replace multiplies by short constants with a series of shifts and adds. Because one bit to the left represents a number twice as large in base 2, shifting the bits left has the same effect as multiplying by a power of 2.
 - Almost every compiler will perform the strength reduction optimization of substituting a left shift for a multiply by a power of 2.
-
- $2 * 4 = 8$
 - $0010 * 0100 = 1000$
 - $0010 \ll 2 = 1000$



MESUT OZDAG, PH.D.
MESUT.OZDAG@UCF.EDU

